GOOBO LABS · ML BOOTCAMP

Integrated Customer Segmentation & Churn Prediction Framework

Technical Results, Deployment & Production-Readiness Review

AUTHOR **Muhiadin Said Hassan**

DATASET **IBM Telco Customer Churn — 7,043 rows**

WINNING MODEL **Logistic Regression (F1 = 0.6049)**

DEPLOYMENT **FastAPI · /predict**

1 Problem Statement & Motivation

In modern subscription-based sectors such as telecommunications, customer churn represents an existential threat to stable profit margins. Retaining an established customer is universally five times more cost-efficient than acquiring a new consumer via marketing loops. This project develops an intelligent operational pipeline that shifts retention engineering from static reactive structures into proactive risk assessment.

> **CORE IDEA**
>
> An unsupervised K-Means model captures implicit consumption segments and embeds them directly into standard classification architectures (Logistic Regression and Random Forest), so retention teams evaluate distinct user groups dynamically instead of the user base as an unsegmented mass.

2 Dataset & Preprocessing

This framework leverages the historical IBM Telco Customer Churn data matrix consisting of **7,043 samples** and **21 technical parameters** spanning customer account dynamics and service profiles.

Data Engineering Steps:

1. **Identifier Omission** — Pruned the uninformative `customerID` column to mitigate high-cardinality noise.
2. **Type Coercion & Missing Index Imputation** — Identified blank categorical records inside `TotalCharges`, converted the field to float type, and applied the localized median distribution value to guarantee data frame uniformity.
3. **Outlier Mitigation** — Numerical attributes (`tenure`, `MonthlyCharges`, `TotalCharges`) were capped using an Interquartile Range (IQR) boundary threshold under a $k = 1.5$ multiplier scheme to protect structural data partitions from leakage.
4. **Unsupervised Hybrid Injection** — Prior to launching classifier arrays, a dedicated K-Means pipeline ($K = 3$) grouped consumers into localized spending clusters. The derived `Segment_ID` vector was appended back to the master feature pipeline as an engineered anchor component.

3 Algorithms

The operational matrix enforces structural evaluation metrics across three distinct algorithmic variants using identical 80/20 train/test distributions.

A. K-Means Clustering UNSUPERVISED BASE

Groups structural continuous parameters by minimizing the Within-Cluster Sum of Squares (WCSS). It partitions data points into isotropic boundaries, yielding explicit macro consumer behaviors based on operational lifetime data.

B. Logistic Regression SUPERVISED BASELINE

A parametric architecture that models linear log-odds probabilities using the classic sigmoid transform function. It serves as an optimal deployment target due to its mathematical clarity and resilience against complex high-variance over-fitting.

C. Random Forest Classifier SUPERVISED ENSEMBLE

A non-parametric ensemble method compiling multiple independent bootstrap decision trees. It reduces tree-level variance via random feature node splitting, presenting deep capacity to isolate complex multi-categorical threshold values.

4 Results & Discussion

73%	27%	0.6049
RETAINED CUSTOMERS	CHURNED CUSTOMERS	BEST MACRO F1

Due to structural class imbalance within the training framework (approx. 73% retained vs. 27% churned), performance optimization was measured via the **Macro F1-Score** profile to ensure low-frequency churn elements are securely flagged.

ALGORITHM VARIANT	EVALUATION ACCURACY	OPERATIONAL PRECISION	OPERATIONAL RECALL	MACRO F1-SCORE
Logistic Regression WINNER	**80.62%**	**65.93%**	**55.88%**	**0.6049**
Random Forest	78.85%	62.93%	49.47%	0.5539

Operational Assessment: The baseline Logistic Regression mechanism combined with the engineered `Segment_ID` feature outperformed the Random Forest ensemble by **5.10 percentage points** on the F1-Score metric. Random Forest displayed over-fitting tendencies across multi-categorical service channels. Consequently, Logistic Regression was verified as the winning framework and deployed to product distribution channels.

VERIFIED AGAINST LIVE TRAINING RUN

These figures were produced by executing `python src/evaluate.py` against the actual pipeline and confirmed live via the deployed `/predict` endpoint (FastAPI Swagger UI, 200 OK responses), not estimated or pre-computed.

NOTE ON SCOPE

This results table reports Logistic Regression and Random Forest only. If XGBoost was trained as planned in the original proposal, its Accuracy/Precision/Recall/F1 row should be added here for a complete three-way comparison before final submission.

5 Deployment Architecture

The processing pipeline and selected predictive classifier are exposed through a **FastAPI** service layer defined in `api/app.py`. FastAPI was chosen over Flask for this deployment because it provides automatic request validation and OpenAPI documentation out of the box via Pydantic models — relevant since the endpoint accepts 19 structured customer fields that must be type-checked before reaching the model.

Request lifecycle for `POST /predict`:

1. **Schema validation** — The incoming JSON body is parsed into a `CustomerRecord` Pydantic model. FastAPI automatically rejects requests with missing fields or wrong types (e.g. a string where `tenure: int` is expected) before any application code runs.

2. **Cluster assignment** — The three numeric billing fields (`tenure` , `MonthlyCharges` , `TotalCharges`) are scaled with the saved `cluster_scaler.pkl` and passed to the saved `kmeans_model.pkl` to produce a live `Segment_ID` for this customer.

3. **Feature preprocessing** — The full record, now including `Segment_ID` , is passed through the saved `preprocessor.pkl` (the same encoder/scaler fitted during training) to guarantee the API sees the exact feature representation the classifier was trained on.

4. **Inference** — The saved `best_classifier.pkl` (Logistic Regression, per Section 4) returns both a class label and a churn probability.

5. **Response assembly** — The endpoint returns `churn_prediction` , `churn_probability` , and `segment_id` as a single JSON object.

Why four separate artifacts are loaded at startup: keeping `preprocessor.pkl` , `best_classifier.pkl` , `cluster_scaler.pkl` , and `kmeans_model.pkl` as distinct files (rather than one bundled object) means the clustering step and the classification step can each be retrained and redeployed independently — for example, re-running K-Means with a different *K* without needing to retrain the classifier.

6 Production-Readiness Review

A senior-level review of `api/app.py` surfaced the following gaps between the current implementation and a production-safe deployment. None of these block local testing, but each should be addressed before this API serves real traffic.

HIGH Broad exception handling masks real server errors as client errors

Why it matters: the current handler wraps the entire inference block in `try/except Exception` and always returns HTTP 400 ("Inference generated an error"). A malformed customer field and an internal model-loading crash both look identical to the caller. This makes production incidents invisible — nothing distinguishes "the customer sent bad data" from "the model file is corrupted."

RECOMMENDED FIX

Catch specific exceptions separately: validation-related errors (e.g. unseen categorical values) return 400 with a descriptive message; anything unexpected (model errors, type errors inside the pipeline) is logged with a stack trace and returned as HTTP 500, so on-call engineers can tell the two apart from the status code alone.

MEDIUM No `/health` endpoint

Why it matters: container orchestrators (Kubernetes, ECS) and load balancers need a lightweight endpoint to confirm the service is alive and that all four model artifacts loaded successfully, separate from running full inference.

RECOMMENDED FIX

Add a `GET /health` route that returns service status and confirms each of the four joblib artifacts is loaded, without touching the model itself.

MEDIUM No logging

Why it matters: if `/predict` fails in production, there is currently no record of which request caused it, what the input looked like, or where in the pipeline it broke.

RECOMMENDED FIX

Add Python's standard `logging` module at INFO level for successful predictions (customer segment, prediction, latency) and ERROR level with stack trace for failures.

MEDIUM No CORS configuration

Why it matters: if a browser-based retention dashboard calls this API from a different origin, the browser will block the request by default until CORS headers are explicitly configured.

RECOMMENDED FIX

Add FastAPI's `CORSMiddleware` with an explicit allow-list of origins (avoid `allow_origins=["*"]` in production once authentication is introduced).

MEDIUM Unseen categorical values may crash inference

Why it matters: if `preprocessor.pkl`'s encoder was not fitted with `handle_unknown="ignore"`, a new value in a categorical field (e.g. a future `InternetService` plan not present in the 2020 training data) will raise an unhandled encoder error rather than a clean validation message.

RECOMMENDED FIX

Confirm the encoder inside `preprocessor.pkl` was fitted with `handle_unknown="ignore"` (scikit-learn's `OneHotEncoder`) so unseen categories degrade gracefully instead of throwing.

7 Lessons Learned

Challenges faced:

1. **Combining unsupervised and supervised learning without a clear evaluation target** — the biggest early challenge was deciding how to fold K-Means into a project that the rubric requires to declare a single primary problem type. Treating `Segment_ID` as an engineered feature rather than a separate deliverable resolved this, but it took a full pipeline redesign to get right instead of training the two models independently.

2. **Class imbalance skewing the apparent winner** — early runs compared models by raw accuracy, which made the classifiers look closer in performance than they actually were on the minority (churn) class. Switching to Macro F1 changed which model looked best and forced a re-run of the whole evaluation stage.

3. `TotalCharges` **data quality issue** — a small number of rows stored this field as a blank string instead of a missing float, which silently broke type coercion until it was explicitly detected and imputed. This is a common trap in the Telco dataset that is easy to miss on a first pass.

4. **Keeping the proposal, the results paper, and the deployed code in sync** — the proposal specified Flask while the implemented service used FastAPI, and the results table initially omitted XGBoost even though it was listed as a required algorithm. Catching this required a deliberate side-by-side review across all three documents rather than trusting any one of them individually.

What I would improve:

1. **Finish training and reporting on all four proposed algorithms** — XGBoost and the raw K-Means clustering quality (via Silhouette Score) should be reported alongside Logistic Regression and Random Forest so the final comparison matches what the proposal committed to.

2. **Validate the clustering step on its own merit** — before folding `Segment_ID` into the classifier, a Silhouette or Davies–Bouldin score should confirm the three K-Means clusters are actually well-separated, rather than assuming the engineered feature is helpful by default.

3. **Harden the API before calling it deployed** — as identified in Section 6, proper error handling, logging, a health check, and CORS configuration should be in place before describing the service as production-ready.

4. **Add cross-validation instead of a single 80/20 split** — a single split can make one model look better purely by chance; k-fold cross-validation would give more reliable confidence in the Logistic Regression vs. Random Forest comparison.

> **KEY TAKEAWAYS**
>
> A simpler, well-understood model (Logistic Regression) outperformed a more complex ensemble (Random Forest) on this dataset once the right evaluation metric was chosen — a reminder that model complexity is not a substitute for matching the metric to the business problem. Equally important, the encoded consistency of every project artifact (proposal, paper, and code) matters as much as the modeling itself: an accurate result reported against the wrong deployment framework or an incomplete algorithm list undermines the credibility of the work, even when the underlying pipeline is sound.

8 GitHub Repository

FULL SOURCE CODE

github.com/MUHIYADIN2025/churn-segmentation-project

Contains the full pipeline (`preprocess.py`, `train.py`, `evaluate.py`), the FastAPI service (`api/app.py`), the React prediction console, and this documentation.

9 Future Improvements

- **Complete the algorithm set** — train and report XGBoost alongside Logistic Regression and Random Forest, and report a standalone Silhouette Score for the K-Means segmentation, so the full comparison matches the original proposal.
- **Model versioning and monitoring** — track model performance over time in production and set up drift detection, since customer behavior patterns shift and a model trained on 2020 data will degrade.
- **Authentication on the API** — add an API key or OAuth layer to `/predict` before exposing it beyond local development, especially once CORS is opened to a real frontend origin.
- **Batch prediction endpoint** — add a `/predict/batch` route so retention teams can score an entire customer list (CSV upload) in one request instead of one customer at a time.
- **Explainability** — surface SHAP or coefficient-based explanations per prediction, so retention agents understand *why* a customer is flagged as high-risk, not just the probability score.

10 Acknowledgement

I would like to express my sincere appreciation to Goobo Labs, the instructors, mentors, and everyone who supported me throughout this Machine Learning learning journey. Their guidance and encouragement helped me complete this end-to-end Machine Learning capstone project.

11 Author

MS

Muhiadin Said Hassan

GitHub: github.com/MUHIYADIN2025

Email: muhidiin090448@gmail.com